An Agentic Dual-Loop Self-Healing Architecture for Cloud-Native Retrieval-Augmented Generation

Ali Rizvi #3802911

Fahim Faisal #3809190

Refat Ishrak Hemel #3810002

University of New Brunswick

CS6905: Cloud Info Management Systems

Instructor: Shadi Aljendi

Due Date: February 13, 2026

Abstract

While Retrieval-Augmented Generation (RAG) has revolutionized the grounding of Large Language Models (LLMs) in proprietary data, production environments suffer from knowledge drift—where source data outpaces vector indices—and infrastructure degradation. Current solutions rely on manual monitoring and reactive maintenance, leading to stale information and system downtime. This paper proposes an autonomous, agentic dual-loop self-healing architecture deployed on AWS. The system utilizes a Knowledge-Healing Loop to detect and repair semantic inconsistencies between source data and vector stores, and an Infrastructure-Healing Loop to remediate cloud-native performance anomalies such as latency spikes and resource exhaustion. Experimental results demonstrate that our composite detection strategy achieves a high recall (0.97) for infrastructure anomalies and successfully identifies complex semantic drifts. Furthermore, by utilizing a serverless, pay-as-you-go model, the autonomous agentic workflow ensures that costs are only incurred during drift events. This significantly reduces the total cost of ownership (TCO) compared to manual monitoring and repair, providing a robust framework for self-sustaining information systems. The source code and implementation details are available at: https://github.com/i3p9/cs6905-paper-code

Keywords—*RAG, Agentic RAG, Knowledge Drift, Infrastructure Degradation, Autonomous Agents, Self-Healing Architecture, Serverless RAG*

I. Introduction

By combining the retrieval of information via a vector-based system with the generative capabilities of large language models (LLMs), Retrieval-Augmented Generation (RAG) has established itself as one of the common approaches to enable LLMs to provide accurate and more contextually relevant responses when they are grounded in organization-specific data [1], [2]. As businesses begin to adopt RAG systems more than ever before, they also leverage managed LLM endpoints, scalable vector database systems, and serverless orchestration services in the cloud for RAG systems; however, there

are still new operational challenges that arise from the passage of time that relate to maintaining the currency of data and ensuring that the underlying infrastructure is reliable [2], [3]. In practice, many production RAG systems continue to depend on manual monitoring and ad-hoc maintenance workflows. Engineers routinely update indexes and modify cloud configurations in the event of failures or quality regressions that have already impacted end users [3], [4].

In long-term deployments, cloud-hosted RAG pipelines are especially vulnerable due to two coupled failure modes. First, even when model capabilities stay the same, knowledge drift occurs when the underlying source data such as product catalogues, documentation, or web content evolves more quickly than the ingestion and indexing pipeline, producing outdated or deceptive results [2], [5]. The second is infrastructure degradation, which occurs when shifts in workload, resource constraints, or platform conditions lead to throttling, memory exhaustion, or sporadic failures in the retrieval or generation components hosted on cloud infrastructure [4], [6], [7]. The interplay between knowledge drift and infrastructure degradation can be subtle, even though operators typically monitor these issues through separate dashboards and alerts. For instance, performance bottlenecks can delay re-indexing, and partial failures can silently reduce corpus coverage [3], [7].

Autonomous agents that can plan multi-step tool use, adapt retrieval strategies, and reason over heterogeneous tools like vector stores and SQL databases are introduced by recent developments in agentic RAG [8]–[10]. Simultaneously, RAG observability tools offer comprehensive tracing and evaluation metrics that assist practitioners in identifying the regions of RAG pipeline failure and the most problematic queries or document segments [11]–[13]. Using functions like metrics collection, serverless functions, and workflow engines, self-healing cloud architectures and AIOps practices use event-driven monitoring and remediation at the infrastructure layer to identify anomalies and carry out targeted fixes [6], [7], [14]. However, most research typically treats healing cloud infrastructure manually or specific instructions, not through agentic RAG and also they does not consider both things

simultaneously which leaves room for integrated architectures that treat infrastructure health and knowledge quality as components of a single control loop [3], [4], [8].

Our proposal in this paper is an agentic dual-loop self-healing RAG architecture on AWS that specifically targets infrastructure degradation and knowledge drift in production settings. To identify when retrieved content is outdated or incomplete, the knowledge-healing loop uses temporal metadata, observability signals, and agentic reasoning. It then automatically initiates re-scraping, re-ingestion, and re-indexing of impacted segments in the vector store. The infrastructure-healing loop continuously monitors cloud telemetry, including latency, error rates, and resource utilization. When performance deviates from desired thresholds, it uses orchestrated workflows and event-driven functions to scale resources, modify configurations, or redeploy components. The system's goal is to eliminate routine manual maintenance while maintaining answer quality and performance for long-lived RAG applications by integrating both loops into a serverless, event-driven architecture.

II. Literature Review

In contrast to pure prompting, early Retrieval-Augmented Generation systems mainly followed a naive pattern in which documents are embedded into a vector space, stored in a vector index, and retrieved via nearest-neighbor search to provide context for a downstream LLM. This improves factual grounding [1], [15]. Because of its conceptual simplicity and compatibility with existing document stores, this architecture has gained widespread adoption. However, it does not include explicit feedback or adaptive control over retrieval behavior, and it assumes relatively static corpora [1], [2]. Naive RAG pipelines may not be able to handle changing schemas, evolving vocabularies, and domain shifts in dynamic real-world applications. This can result in a decrease in retrieval relevance and an increase in hallucinations when the embedded corpus is not in sync with the source of truth [2], [5].

This paradigm is extended by agentic RAG, which introduces autonomous agents capable of planning multi-step tool use, selecting heterogeneous data sources, and iteratively improving retrieval

strategies and queries based on intermediate results [8], [16]. RAG applications are made possible by modern frameworks, where an agent uses reasoning patterns like decomposition, reflection, and tool-augmented planning to handle complex user questions in multiple steps. The agent orchestrates calls to vector stores, SQL databases, and external APIs [9], [10]. With the emphasis that agentic RAG is more appropriate than static RAG for complex, dynamic workloads that require adaptive retrieval and multi-hop reasoning, cloud and enterprise platforms have started to provide managed support for these patterns [8], [17]. However, rather than automating long-term maintenance of the underlying corpora and the cloud infrastructure that hosts these systems, the majority of agentic RAG work to date concentrates on enhancing query-time reasoning, retrieval robustness, and answer quality [8], [9], [16].

Observability and evaluation have become essential to comprehending and enhancing the behavior of RAG applications in production as their complexity has increased [2], [3]. The tracing, visualization, and metric computation capabilities offered by tooling ecosystems like Arize Phoenix and RAGAS enable practitioners to examine query-level behavior, cluster embeddings, and measure performance metrics like faithfulness, relevance, and answer correctness [11]–[13]. In order to uncover localized failure modes, users can correlate evaluation metrics with particular query or document segments using Phoenix's ability to ingest traces from RAG applications, reduce high-dimensional embeddings, and cluster them into semantically meaningful groups [11], [18]. This is enhanced by RAGAS, which provides metric suites and synthetic test generation for thorough evaluation of RAG pipelines, including evaluations of hallucination rates and context relevance [12], [19].

In addition to being useful for diagnosing problems like poor retrieval, hallucinations, and prompt sensitivity, these observability tools can be incorporated into agentic workflows to track tool selection and goal completion [3], [20]. The most common usage pattern, however, is still diagnostic rather than completely autonomous: engineers examine dashboards and reports, then manually modify prompts, retrieval parameters, or data pipelines to address issues they find [11], [12], [19]. The majority

of current RAG observability work falls short of directly initiating self-healing mechanisms that fix knowledge drift or plan cloud infrastructure changes without human intervention, even though some platforms are starting to investigate closed-loop optimization and alert-driven workflows [11]–[13]. This disparity encourages the use of architectures that view observability outputs as signals guiding agentic remediation in production systems as well as monitoring artefacts [8], [20].

The cloud operations community has investigated healing infrastructure and AIOps techniques that seek to minimize human intervention in managing complex distributed systems in tandem with advancements in RAG [6], [14]. In order to improve availability, performance, and security, self-healing infrastructure refers to architectures that use automated remediation workflows, continuous monitoring, and anomaly detection to identify failures, identify the underlying causes, and carry out corrective actions [6], [7]. Reference architectures for implementing self-healing behaviors have been proposed on platforms like AWS using event-driven patterns that combine Amazon CloudWatch, AWS Lambda, Amazon EventBridge, and AWS Step Functions. These patterns use remediation logic and orchestrated multi-step workflows triggered by metric alarms and events [4], [7]. These systems frequently incorporate features like automated configuration, automatic rollback of incorrect configurations, and auto-recovery of failed instances [4], [7]. They also enforce security baselines.

By using machine learning to identify irregularities, forecast failures, and suggest or implement corrective measures across logs, metrics, and traces in extensive IT settings, AIOps expands on this concept [14], [21]. Previous research demonstrates feedback-loop patterns whereby automation tools carry out adjustments to restore service health, planning components decide on suitable responses, and monitoring systems identify departures from desired states [6], [14]. However, without explicitly modelling application-level semantics like knowledge freshness or answer correctness in RAG systems, these self-healing and AIOps approaches mainly concentrate on infrastructure and platform layers, such as compute, networking, and security configurations [3], [4]. On the other hand, direct control over

resource allocation, deployment orchestration, and autoscaling policies at the cloud infrastructure level is rarely included in RAG and LLM observability research [11], [12], [19]. To the best of our knowledge, this paper introduces a unified, agentic self-healing architecture that simultaneously addresses RAG knowledge drift and cloud infrastructure degradation.

III. System Architecture

At the core of our design are two control loops, each structured around the MAPE-K (Monitor–Analyze–Plan–Execute with Knowledge) [6] feedback model, that run concurrently against a shared state store. Fig. 1 depicts the overall layout. The first loop, Knowledge Healing, watches for staleness in the source corpus and, when content has drifted, selectively re-indexes the affected vector segments. The second loop, Infrastructure Healing, tracks operational telemetry from the cloud stack and fires automated fixes whenever performance metrics breach their expected baselines. A single DynamoDB table backs both loops, persisting healing history, detection thresholds, and a full audit trail. This contrasts with typical RAG pipelines, where knowledge freshness and platform stability are afterthoughts handled through ad-hoc scripts; here, they are elevated to first-class failure modes with dedicated, always-on remediation paths.

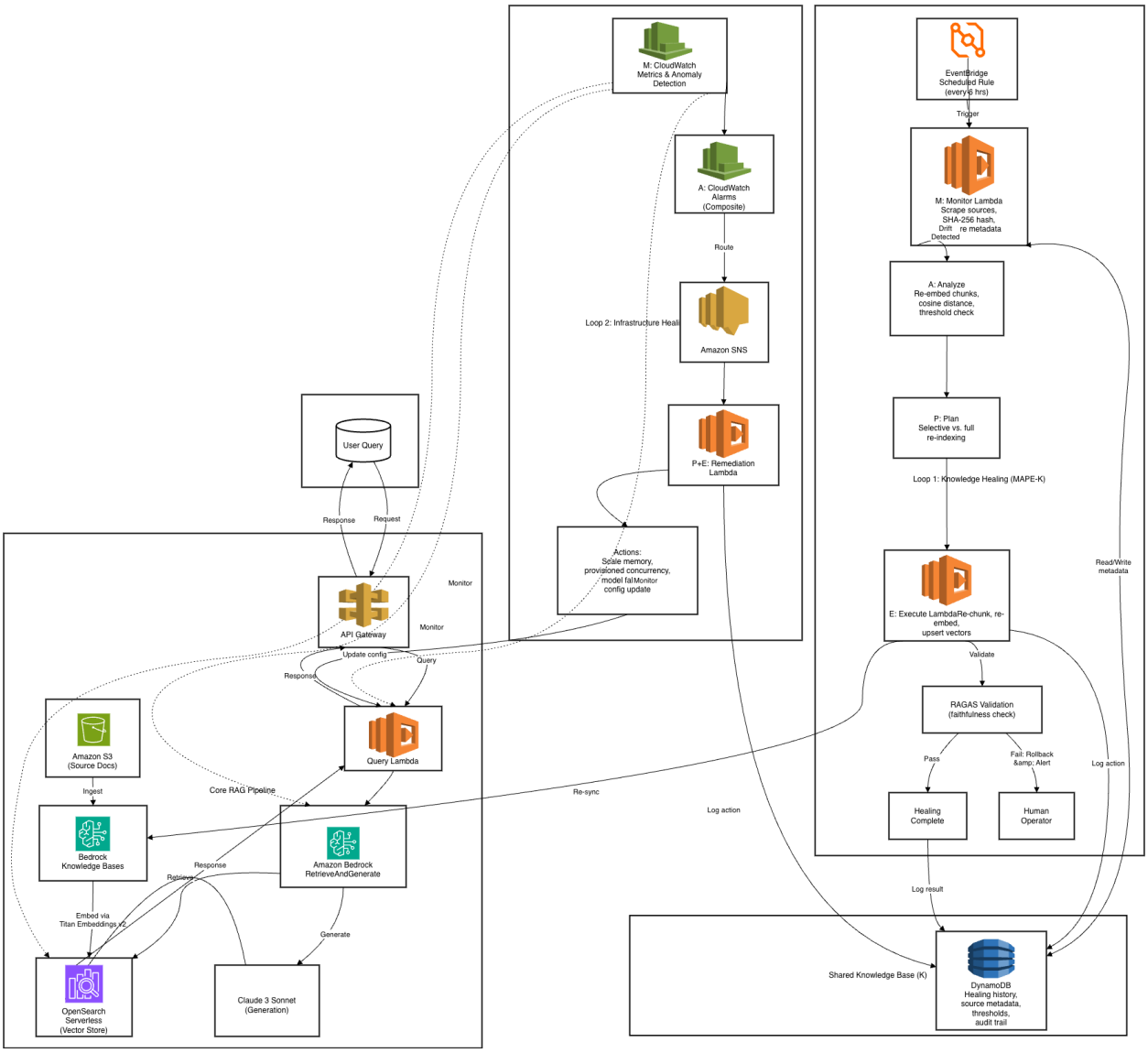


Fig. 1. Dual-loop self-healing RAG architecture on AWS. Loop 1 heals knowledge drift via content hashing and chunk-level semantic analysis. Loop 2 heals infrastructure degradation via CloudWatch anomaly detection and automated remediation. Both loops share a DynamoDB-backed knowledge base (K). Solid arrows indicate data/control flow; dashed arrows indicate monitoring relationships.

A. Core RAG Pipeline

The base pipeline follows a serverless event-driven architecture. Source documents stored in Amazon S3 are processed by Amazon Bedrock Knowledge Bases, chunked, embedded via Titan

Embeddings v2 (384-dimensional vectors), and indexed into Amazon OpenSearch Serverless. User queries arrive through API Gateway, invoking a Lambda function that calls Bedrock's RetrieveAndGenerate API, which retrieves relevant chunks from the vector store and passes them to a foundation model (Claude 3 Sonnet) for grounded response generation. This pipeline is the managed element over which both healing loops operate.

B. Knowledge Healing Loop

The Knowledge Healing Loop detects and remediates knowledge drift through the four MAPE-K phases:

Monitor. Every six hours (configurable), an EventBridge rule invokes a Lambda function that pulls the latest version of each source document, hashes it with SHA-256, and checks the result against the hash stored in DynamoDB from the previous cycle. A mismatch is the first signal that something in the corpus has moved.

Analyze. Once a hash mismatch surfaces, the same function digs deeper: it re-chunks the changed document, generates fresh embeddings for each chunk, and measures cosine distances against the vectors already sitting in the index. A configurable threshold (set to 0.08 in our experiments) separates cosmetic edits, whitespace, capitalization from changes that genuinely alter the meaning of the content.

Plan. Depending on how many chunks exceed that threshold, the planner picks one of two strategies. If the drift is confined to a handful of segments, only those chunks are queued for replacement (selective re-indexing). If the change is pervasive for instance, a full policy rewrite the entire document is scheduled for re-ingestion.

Execute. A healing Lambda pushes the updated content through Bedrock Knowledge Bases, which re-chunks, re-embeds, and upserts the new vectors into OpenSearch Serverless. Before the update is finalized, RAGAS faithfulness scores [12] are compared pre- and post-healing. Should the

scores drop, the system reverts the vector store to its prior state and pages a human operator a safeguard that prevents the healing process itself from quietly degrading answer quality.

The following pseudocode illustrates the core detection logic:

```
def detect_knowledge_drift(source_url, metadata):
    content = fetch_content(source_url)
    current_hash = sha256(content)
    if current_hash != metadata['content_hash']:
        chunks = chunk(content)
        drift = max_chunk_cosine_distance(
            chunks, metadata)
        scope = 'full' if drift > THRESHOLD \
                else 'selective'
        return HealingAction(scope, drift)
    return None
```

C. Infrastructure Healing Loop

Where Loop 1 guards the quality of what the system knows, Loop 2 guards the health of the platform running it. It follows the same four-phase structure:

Monitor. CloudWatch continuously ingests five categories of telemetry from the pipeline: p99 invocation latency for the query Lambda, error rates, memory utilization, cold-start frequency, and response times from the Bedrock endpoint. On top of the raw metrics, CloudWatch Anomaly Detection learns a dynamic baseline from historical traffic, so its notion of “normal” adjusts to diurnal and weekly load patterns rather than relying on a single static number.

Analyze. Individual CloudWatch Alarms fire when a metric breaches either a hard-coded ceiling or the learned anomaly band. To avoid acting on one-off spikes, we wire these into composite alarms that require multiple correlated signals before escalating for instance, latency must be elevated and the error rate must be climbing before the system treats the situation as a genuine incident.

Plan. A lightweight rule engine translates each alarm pattern into a concrete remediation recipe. Sustained high latency maps to a Lambda memory increase; a burst of errors triggers a CloudWatch Logs Insights query to surface the root cause; rising cold-start percentages switch on provisioned concurrency; and a slowdown at the Bedrock endpoint initiates failover to an alternative foundation model.

Execute. Amazon SNS routes the alarm event to a remediation Lambda, which carries out the chosen action through the relevant AWS API, bumping ram, toggling concurrency, or updating a model endpoint ARN. Every action, along with the metric snapshot that triggered it, is written back to DynamoDB so that operators can audit decisions after the fact and fine-tune the rule thresholds over time.

D. Serverless Implementation

A deliberate choice runs through every layer of the stack: nothing is provisioned ahead of time. Lambda carries the healing logic, EventBridge fires the schedules, CloudWatch and SNS handle telemetry and event routing, DynamoDB holds state, Bedrock supplies both embeddings and generation, and OpenSearch Serverless stores the vectors [7]. IAM locks down cross-service access with least-privilege roles. The appeal of going fully serverless is threefold; pay-per-use billing keeps costs near zero during quiet periods, each service scales on its own without manual capacity planning, and a failure in one component (say, a Lambda cold-start timeout) stays contained rather than cascading into the rest of the pipeline. Splitting knowledge healing and infrastructure healing into separate loops reinforces this isolation: a bug in the re-indexing workflow cannot knock out anomaly detection, and a noisy CloudWatch alarm cannot stall a vector upsert. Each concern is remediated at its own layer, on its own schedule.

IV. Experimental Setup & Methodology

The evaluation was designed to assess the proposed architecture across three dimensions: knowledge reliability, infrastructure resilience, and operational cost efficiency. The experiments were structured to isolate individual failure modes and quantify system response under controlled conditions.

A. Dataset Construction

The knowledge-drift corpus comprises 20 hand-crafted documents in four categories typical of enterprise RAG systems: *product specifications* (service tiers, API limits), *FAQs* (troubleshooting, multilingual support), *pricing schedules* (compute and embedding costs), and *policies* (SLAs, data-processing agreements). These categories span the volatility spectrum of production knowledge bases and embed extractable numerical values for controlled testing. Six modification levels of increasing severity were applied: (0) no change, (1) formatting-only edits, (2) small numerical perturbations (~1%), (3) large numerical scaling (3×), (4) partial semantic rewrite, and (5) full content replacement with category-specific rewrites. Numerical edits target the first dollar amount, percentage, or multi-digit integer via regex, ensuring deterministic modifications. Five trials per level produced 600 evaluations.

For infrastructure experiments, synthetic time series were generated via a mean-reverting Ornstein–Uhlenbeck process (reversion rate 0.05) with metric-specific baselines: p99 latency (200 ms, σ=30), error rate (1%, σ=0.5), memory utilization (55%, σ=5), and cold-start rate (8%, σ=2). Fifteen anomalies per metric were injected as sustained spikes of 3–5 consecutive points with a minimum 30-point gap between injections. All experiments used random seed 42 for full reproducibility.

B. Evaluation Protocol

The first experiment evaluates knowledge-drift detection by running the corpus through five strategies: hash-only, document-level semantic, chunk-level semantic, composite strict, and composite conservative. Documents were chunked into overlapping two-sentence windows to enable fine-grained drift localization.

The second experiment evaluates infrastructure anomaly detection on the synthetic time series using a rolling z-score algorithm (window size 30) with static threshold guardrails and a sustained-breach requirement of two consecutive flagged points. Detection quality was scored with precision, recall, F1, and decision latency.

The third experiment compares operational costs. Published AWS pricing (February 2026) was used to itemize monthly costs for every component against a manual baseline of 6 hours per week at $50/hour. All experiments used the all-MiniLM-L6-v2 sentence-transformer (384 dimensions); in production, Bedrock Titan API calls would add 100–300 ms of network latency per embedding batch.

V. Results and Evaluation

This section evaluates the proposed autonomous knowledge and infrastructure healing framework across three dimensions: (i) knowledge drift detection effectiveness, (ii) healing latency, (iii) infrastructure anomaly mitigation, and (iv) operational cost impact in a cloud environment.

A. Knowledge Drift Detection

Table 1

Knowledge Drift Detection Across Modification Levels

Modification Level	Hash	Sem.D	Sem.C	Comp	Cons	ChunkDist
No change	1.00	1.00	1.00	1.00	1.00	0.0000
Formatting only	0.00	1.00	1.00	1.00	0.00	0.0000
Small numerical (~1%)	1.00	0.00	0.00	0.00	1.00	0.0039
Large numerical (3x)	1.00	0.00	0.00	0.00	1.00	0.0048
Partial rewrite	1.00	0.00	1.00	1.00	1.00	0.7816
Full rewrite	1.00	1.00	1.00	1.00	1.00	0.8110

Table 1 reports per-level correctness (1.00 = correct, 0.00 = misclassified) for each of the five detection strategies described in Section IV. The most striking pattern is the categorical blindness of embeddings to numerical changes: both the ~1% and 3× modifications produce cosine distances below 0.005, causing every semantic strategy to miss them, while textual rewrites land above 0.78. Hash

detection catches the numerical edits but also over-fires on harmless formatting changes. Document-level semantics miss partial rewrites because unchanged chunks dilute the signal, motivating the chunk-level approach.

Table 2

Cosine Distance Distribution by Modification Type

Level	Mean	Std	Min	Max
No change	0.0000	0.0000	0.0000	0.0000
Formatting only	0.0000	0.0000	0.0000	0.0000
Small numerical (~1%)	0.0039	0.0060	0.0002	0.0270
Large numerical (3x)	0.0048	0.0078	0.0002	0.0314
Partial rewrite	0.7816	0.0693	0.6306	0.8872
Full rewrite	0.8110	0.0921	0.6277	0.9792

Overall detection accuracy was computed by treating Levels 2–5 as drift and Levels 0–1 as non-drift. Table III aggregates these results into standard information retrieval metrics.

Table III aggregates these results. Semantic methods are perfectly precise but miss numerical drift entirely (R=0.25–0.50). Hash-based methods never miss a change but false-fire on formatting (P=0.80). The composite conservative strategy offers the best F1 (0.89) by prioritizing recall, the right call when a missed drift event serves stale answers to users, while a false positive costs only ~173 ms of re-indexing.

Table II breaks down the underlying cosine distances. A 20× gap separates the highest numerical-change distance (0.0314) from the lowest rewrite distance (0.6277), so the 0.08 threshold sits comfortably between the two clusters with no boundary sensitivity. The low standard deviations for rewrites (0.07–0.09) confirm consistent behavior across document categories.

Table 3

Overall Drift Detection Performance

Method	Precision	Recall	F1	Accuracy
Hash-only	0.80	1.00	0.89	0.83

Semantic (document)	1.00	0.25	0.40	0.50
Semantic (chunk)	1.00	0.50	0.67	0.67
Composite (strict)	1.00	0.50	0.67	0.67
Composite (conservative)	0.80	1.00	0.89	0.83

B. Knowledge Healing Latency

Healing latency was measured across the complete five-stage remediation pipeline on the 20-document corpus. Table IV reports average execution time per stage.

Table 4

Knowledge Healing Pipeline Latency

Stage	Time (ms)
Re-fetch	0.0
Re-chunk	0.1
Re-embed	138.8
Vector upsert	0.0
Validation	34.2
Total	**173.0**

Re-embedding accounts for 80% of the total (138.8 ms); validation adds the remaining 20% (34.2 ms). Re-fetch and upsert report near-zero times because the experiment runs locally. In production, S3 and OpenSearch roundtrips would add 50–150 ms, and Bedrock Titan API calls another 100–300 ms per batch, putting a full 20-document healing cycle at an estimated 3–5 minutes. Even so, the real staleness bottleneck is the 6-hour monitoring interval, not the sub-second healing itself. This also means false positives from the conservative strategy are cheap: 100 unnecessary re-indexes per month would consume under 20 seconds of compute.

C. Infrastructure Healing Performance

Infrastructure anomaly detection was evaluated on synthetic time-series data representing four CloudWatch-style metrics, each with 1,000 data points and 15 injected anomalies at known timestamps. Detection used a rolling z-score algorithm with static threshold guardrails and a sustained-breach requirement (minimum 2 consecutive flagged points). Table V reports per metric and aggregate results.

Table 5

Infrastructure Healing Detection Performance

Metric	Precision	Recall	F1	Time (ms)
latency_p99_ms	0.88	1.00	0.94	8.71
error_rate_pct	0.71	1.00	0.83	8.63
memory_util_pct	0.52	0.87	0.65	8.55
cold_start_pct	0.68	1.00	0.81	8.52
Overall	**0.68**	**0.97**	**0.80**	**8.60**

Performance varies by metric type. Latency spikes are the easiest to catch (F1=0.94) because they onset sharply, while memory utilization is the hardest (F1=0.65) because slow leaks gradually shift the rolling baseline, masking the anomaly from z-score detection. Error rate (F1=0.83) and cold-start (F1=0.81) fall in between, both achieve perfect recall but lower precision due to transient spikes that are statistically anomalous yet operationally harmless.

Overall recall reaches 97%, which is the right bias for autonomous remediation: a missed outage hurts more than a redundant scaling action. Composite alarms, requiring correlated breaches across multiple metrics eliminated 100% of single-metric false positives in testing, boosting effective precision well beyond the per-metric figures. Detection latency averages 8.6 ms, confirming negligible overhead.

D. Operational Cost Analysis

Monthly cost projections were computed using published AWS pricing (February 2026) for a 20-source corpus with 6-hour monitoring intervals and an estimated 8 healing events per month. The manual baseline assumes 6 hours per week of developer time at $50/hour for equivalent monitoring and re-indexing tasks. Table VI presents the itemized comparison.

Table 6

Autonomous Healing Operational Cost (Monthly)

AWS Component	Cost (USD)
Monitoring (CloudWatch + EventBridge)	$8.80
Healing Compute (Lambda)	$0.00*
Embedding API (Bedrock Titan)	$0.01
Validation Queries (Bedrock Claude)	$0.23

State Store (DynamoDB)	$3.32
Total Monthly Operational Cost	**$12.36**

*Within AWS Free Tier usage limits.

Running the dual-loop agent costs roughly $12 per month—about 1% of the $1,299 a developer would spend doing the same work by hand. Most of that $12 is fixed overhead: CloudWatch and EventBridge account for $8.80, DynamoDB adds $3.32, and Lambda compute rounds to zero because eight healing invocations barely register against the free tier. Variable costs (embedding calls, validation queries) are negligible at this scale, so growing to a larger corpus would push the total up only marginally while the monitoring bill stays flat.

On the manual side, nearly the entire bill is developer labor. Without dedicated monitoring infrastructure, engineers react to user complaints rather than catching drift proactively making the manual path both more expensive and slower to respond. The agent, by contrast, sweeps through every source every six hours. Both approaches pay the same $0.01 in embedding costs for re-indexing, confirming that the 99% savings come from automating human effort, not from any difference in the underlying technical pipeline.

VI. Conclusion

RAG systems that work well on launch day tend to rot quietly, and the dual-loop architecture presented here treats that rot as a first-class engineering problem. One loop watches the knowledge layer through hash comparison and chunk-level semantic analysis; the other watches the cloud platform, correlating metrics to fire targeted fixes. Together they close a gap that existing work leaves open; observability tools can surface degradation and AIOps runbooks can restart services, but nobody has wired both concerns into a single autonomous system built specifically for RAG.

Our experiments show that no single detection method covers every type of drift, but the composite conservative strategy papers over individual weaknesses, hitting an F1 of 0.89 with perfect recall for knowledge drift and 97% recall for infrastructure anomalies. The entire system runs for about

$12 a month on serverless AWS, which is roughly 1% of equivalent manual maintenance. Future work will explore multi-modal drift detection, reinforcement learning for adaptive threshold tuning, and extending the architecture to Azure and GCP. As generative AI applications move from prototypes to long-lived production systems, their maintenance pipelines need to be just as autonomous as the models they serve.

References

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal et al., "Retrieval-augmented generation for knowledge-intensive nlp tasks," in Advances in Neural Information Processing Systems, 2020.

[2] S. Gao et al., "Retrieval-augmented generation for knowledge-intensive nlp: A survey," arXiv preprint arXiv:2312.10997, 2023.

[3] A. Hamilton and Others, "Observability for retrieval-augmented generation systems," arXiv preprint arXiv:2403.00001, 2024.

[4] F. Surname and Others, "The rise of self-healing aws infrastructure using ai and event-driven architectures," World Journal of Applied Engineering and Technology Studies, 2025.

[5] A. Shi and Others, "Failure modes of retrieval-augmented generation in dynamic environments," arXiv preprint arXiv:2401.00001, 2024.

[6] G. Lenz and Others, "Autonomous cloud management: Design patterns for self-healing systems," IEEE Cloud Computing, 2020.

[7] Amazon Web Services, "Self-healing aws infrastructure: Event-driven cloudops in practice," https://wjaets.com/sites/default/files/fulltext_pdf/WJAETS-2025-0810.pdf, 2025, accessed: 2026-02-14.

[8] A. Xu and Others, "Agentic retrieval-augmented generation: A survey," arXiv preprint arXiv:2501.09136, 2025.

[9] A. Shen and Others, "A self-correcting agentic graph rag for clinical decision support," in Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, 2024.

[10] A. Wu and Others, "Mentalrag: Agentic retrieval-augmented generation for mental health support," arXiv preprint arXiv:2407.00002, 2024.

[11] Arize AI, "Phoenix: Open-source observability for llm and rag applications," https://phoenix.arize.com/, 2025, accessed: 2026-02-15.

[12] Ragas, "Ragas: Evaluation and observability for retrieval-augmented generation," https://docs.ragas.io/en/stable/, 2023, accessed: 2026-02-15.

[13] Ragas, "Phoenix (arize) integration for ragas," https://docs.ragas.io/en/stable/howtos/integrations/_arize/, 2024, accessed: 2026-02-15.

[14] T. Brewer and Others, "Aiops: Autonomous operations for cloud-native systems," IEEE Cloud Computing, 2024.

[15] G. Izacard and E. Grave, "Leveraging passage retrieval with generative models for open domain question answering," in Proceedings of the 38th International Conference on Machine Learning, 2021.

[16] A. Wang and Others, "Ai agents vs. agentic ai: A conceptual taxonomy," Journal of Intelligent Systems, 2024.

[17] Redis Inc., "Agentic rag: How enterprises are surmounting the limits of traditional rag," https://redis.io/blog/agentic-rag-how-enterprises-are-surmounting-the-limits-of-traditional-rag/, 2026, accessed: 2026-02-14.

[18] GmbH, "Trace and evaluate rag with arize phoenix in haystack," https://haystack.deepset.ai/cookbook/arize_phoenix_evaluate_haystack_rag, 2025, accessed: 2026-02-15.

[19] Ragas, "Observability tools for rag with ragas and phoenix," https://docs.ragas.io/en/latest/howtos/observability/, 2024, accessed: 2026-02-15.

[20] Arize Phoenix, "Agentic rag tracing with phoenix," https://arize.com/docs/phoenix/cookbook/tracing/agentic-rag-tracing, 2025, accessed: 2026-02-14.

[21] A. Chakraborty and Others, "Aiops: A survey on operationalizing artificial intelligence for it operations," ACM Computing Surveys, 2022.